

OBJECT ORIENTED PROGRAMMING

S.Gayathri

Assistant Professor,

Department of Computer Science and Engineering

Jerusalem College of Engineering

UNIT-I BASICS OF C++

Object Oriented Programming: Concepts of Object-Oriented Programming - Origins of C++ — Keywords - Classes and Objects — Constructors — Destructors — Static Data Members - Static Member Functions — Arrays Fundamentals — String Operations.

Principles of Object Oriented Programming

- Objects
- Classes
- Encapsulation
- Data abstraction
- Polymorphism
- Inheritance
- Dynamic binding
- Message passing

Objects:

Objects are the basic run-time entities in an object oriented system. They may represent a person, place, bank account etc.

Classes:

- Objects contain data and code to manipulate that data.
- Objects are variables of the type class.
- A class is a collection of objects of similar type

Encapsulation: Wrapping of data and functions together as a single unit is known as encapsulation.

- By default data is not accessible to outside world and they are only accessible through the functions which are wrapped in a class.
- prevention of data direct access by the program is called **data hiding or information hiding**

Data abstraction :

- Abstraction refers to the act of representing essential features without including the back ground details or explanation.

Polymorphism:

- Polymorphism comes from the Greek words “poly” and “morphism”. “poly” means many and “morphism” means form i.e.. many forms.
- Polymorphism means the ability to take more than one form.
- For example, an operation have different behavior in different instances. The behavior depends upon the type of the data used in the operation.

Inheritance:

- Inheritance is the process by which one object can acquire the properties of another.

Dynamic binding:

- Binding refers to linking of procedure call to the code to be executed in response to the call. Dynamic binding(or late binding) means the code associated with a given procedure call is not known until the time of call at run time.

Message passing:

- An object oriented program consists of a set of objects that communicate with each other.
- Objects communicate with each other by sending and receiving information.
- A message for an object is a request for execution of a procedure and therefore invokes the function that is called for an object and generates a result.

- **Benefits of object oriented programming (OOPs)**

- **Reusability:** In OOP's programs functions and modules that are written by a user can be reused by other users without any modification.
- **Inheritance:** Through this we can eliminate redundant code and extend the use of existing classes.
- **Data Hiding:** The programmer can hide the data and functions in a class from other classes. It helps the programmer to build the secure programs.
- **Reduced complexity of a problem:** The given problem can be viewed as a collection of different objects. Each object is responsible for a specific task. The problem is solved by interfacing the objects. This technique reduces the complexity of the program design.
- **Easy to Maintain and Upgrade:** OOP makes it easy to maintain and modify existing code as new objects can be created with small differences to existing ones. Software complexity can be easily managed.
- **Message Passing:** The technique of message communication between objects makes the interface with external systems easier.

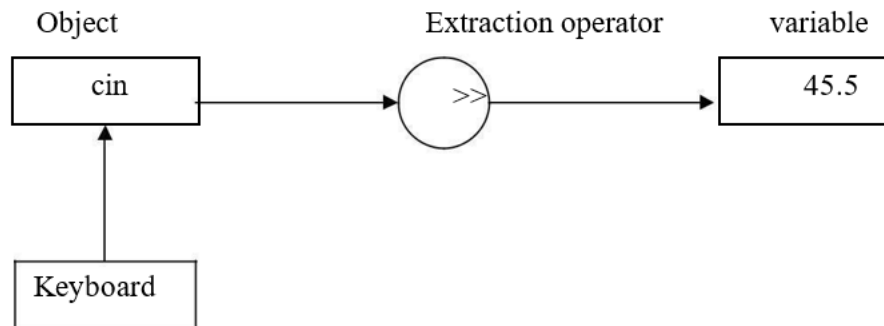
BASIC STRUCTURE OF C++ LANGUAGE :

- The program written in C++ language follows this basic structure.

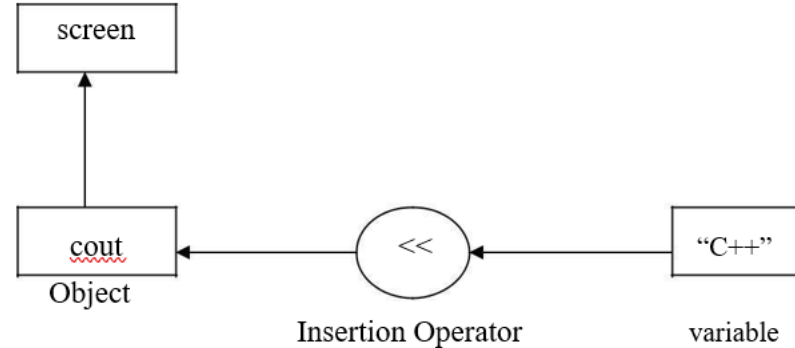
1. Documentation section
2. Linking section
3. Definition section
4. Global declaration section & class declarations
5. Member function definition
6. Main function
 - section main()
{
Declaration section
Executable section
}

Input-Output Statements

- The statement `cin>>n;` is an input statement and causes the program to wait for the user to type in a number. The number keyed is placed on the variable “n”. The identifier `cin` is a predefined object in C++ that corresponds to the standard input stream. The operator `>>` is known as extraction operator. It extracts the value from the keyboard and assigns it to the value variable on its right.



- `cout` is a predefined object that represents the standard output stream. The operator `<<` is an insertion operator, causes the string in double quotes to be displayed on the screen.



Key Words

Tokens

The smallest individual units in a program are known as tokens.

C++ has the following tokens:

- Keywords
- Identifiers
- Constants
- Strings
- Operators

Keywords

- They are explicitly reserved identifiers.

Example: auto, break, public, private, protected, continue, delete, catch, try, virtual, inline, friend, union, struct, int, if etc.

48 keywords and additional keywords are available.

Class and Objects

- **class:** It is a collection of data and member functions that manipulate data. The data components of class are called data members and functions that manipulate the data are called member functions.

- Syntax:-

```
class class_name  
{
```

Access specifier :

Variable declarations;

Access specifier :

function declarations;

```
};
```

- **Access Control:**
 - **Access specifier or access modifiers** are the labels that specify type of access given to members of a class. These are used for data hiding. These are also called as visibility modes. There are three types of
1.private 2.public 3.protected

- **Private:**

- If the data members are declared as private access then they cannot be accessed from other functions outside the class. It can only be accessed by the functions declared within the class. It is declared by the key word „**private**“ .

public:

- If the data members are declared public access then they can be accessed from other functions outside the class. It is declared by the key word „**public**“ .

protected: The access level of protected declaration lies between public and private. This access specifier is used at the time of inheritance

Note:-

- If no access specifier is specified then it is treated by default as private

Example:

```
class student
```

```
{
```

```
private :
```

```
    int roll;
```

```
    char name[30];
```

```
public:
```

```
    void get_data()
```

```
    {
```

```
        cout<<"Enter roll number and name":
```

```
        cin>>roll>>name;
```

```
    }
```

```
    void put_data()
```

```
    {
```

```
        cout<<"Roll number:"<<roll<<endl;
```

```
        cout<<"Name   :"<<name<<endl;
```

```
    }
```

```
};
```

Object:-Instance of a class is called object.

Syntax:

- `class_name object_name;`

Ex:

- `student s;`

Accessing members:-dot operator is used to access members of class

Object-name.function-name(actual arguments);

Ex:

`s.get_data();`

`s.put_data();`

Program:

```
#include<iostream>

class student
{
    private:
        int roll;
        char name[20];

    public:
        void getdata()
        {
            cout<<"Enter Roll number:";
            cin>>roll;
            cout<<"Enter Name:";
            cin>>name;
        }
        void putdata()
        {
            cout<<"Roll no:"<<roll<<endl;
            cout<<"Name:"<<name<<endl;
        }
};
```

```
int main()
{
    student s;
    s.getdata();
    s.putdata();
    return 0;
}
```

Output:

Enter Roll number:

101

Enter Name

Ajay

Roll no:101

Name:Ajay

Defining member Functions

- Outside the class definition
- Inside the class definition

Outside

Return-type class-name :: function name(argument declaration)

{

Body of function

}

Program: Declaring Function outside the class

```
#include<iostream>
```

```
class student
```

```
{
```

```
    private:
```

```
        int roll;
```

```
        char name[20];
```

```
    public:
```

```
        Void getdata();
```

```
        Void putdata();
```

```
};
```

```
void student::getdata()
```

```
{
```

```
    cout<<"Enter Roll number:"; cin>>roll;
```

```
    cout<<"Enter Name:"; cin>>name;
```

```
}
```

```
void student:: putdata()
{
    cout<<"Roll no:"<<roll<<endl;
    cout<<"Name:"<<name<<endl;
}
int main()
{
    student s;
    s.getdata();
    s.putdata();
    return 0;
}
```

Output:

Enter Roll number:

101

Enter Name

Ajay

Roll no:101

Name:Ajay

Constructors & Destructors

Constructors

- A constructor is a special member function whose task is to initialize the objects of its class.
- Its name is same as the class name.
- The Constructor is invoked whenever an object of its associated class is created.
- It is called constructor because it constructs the values of data members of the class.

Characteristics of the constructors:

- Constructors should be declared in public section.
- They are invoked automatically when the objects are created.
- A constructor will have exact same name as the class and it does not have any return type at all, not even void and they cannot return values.
- They cannot be inherited though a derived class can call the base class constructor.
- A constructor can have default arguments.
- Constructors cannot be virtual & We cannot refer to their addresses.
- They make implicit calls to the operators new and delete when memory allocation is required.

Types of Constructors:

- Default Constructors
- Parameterized constructors
- Copy constructors

Default Constructors:

- A constructor that accepts no parameters is called the default constructor.

Example:

Class integer

```
{  
int m,n;  
public:  
integer(void);  
...  
...  
};  
integer::integer(void)  
{  
    m=0;  
    n=0;  
}
```

Invoking Default constructors:

Class_name object_name;

integer i1; //Default constructor is invoked

ii). Parameterized Constructors:

The constructors that can take arguments are called parameterized constructors.

Example:

Class integer

```
{  
int m,n;  
public:  
integer(int,int);  
...  
};  
integer::integer(int x,int y)  
{  
    m=x;  
    n=y;  
}
```

Invoking Parameterized constructors:

Explicit call:

```
Class_name object_name=class_name(parameters);
```

```
integer i1=integer(10,20); //parameterised constructor is  
invoked
```

Implicit call:

```
Class_name object_name(parameters);
```

```
integer i1(10,20); //parameterised constructor is invoked
```

iii). Copy Constructors:

- A constructor can accept reference to its own class as parameter ,In such cases,the constructor is called as copy constructor.
- This helps to copy member values of one object to other object at the time of its creation.

Example:

Class integer

{

int m,n;

public:

integer(integer&);

...

};

integer::integer(integer &r)

{

m=r.m;

n=r.n;

}

Invoking Copy constructors:

Explicit call:

```
Class_name object_name=object_name;  
        integer i2=i1; //copy constructor is invoked
```

Implicit call:

```
Class_name object_name(object_name);  
        integer i2(i1); //copy constructor is invoked
```

Destructors

- A destructor is used to destroy the objects that have been created by a constructor.
- A destructor will have exact same name as the class prefixed with a tilde (~) and defined in public section.
- It can neither return a value nor can it take any parameters.
- Destructor can be very useful for releasing resources before coming out of the program like closing files, releasing memories etc.

Example :

```
#include<iostream.h>
#include<conio.h>
class alpha
{
int m,n;
public:
alpha()
{
cout<<"default constructor :"<<endl;
m=0;
n=0;
}
alpha(int x,int y)
{
cout<<"parameterized constructor:"<<endl;
```

```
m=x;  
n=y;  
}  
alpha(alpha &a)  
{  
cout<<"copy construtor:"<<endl;  
m=a.m;  
n=a.n;  
}  
~alpha()  
{  
cout<<"Destructor invoked:"<<endl;  
}
```

```
void display()
{
    cout<<"m="<<m<<endl;
    cout<<"n="<<n<<endl;
}

};

void main()
{
    clrscr();
    alpha a1;
    a1.display();
    alpha a2(10,20);
    a2.display();
```

```
alpha a3=a2;  
a3.display();  
getch();  
}
```

Output:

default constructor:

m=0

n=0

parameterized constructor:

m=10

n=20

copy constructor:

m=10

n=20

Destructor invoked

Destructor invoked

Destructor invoked

Static Data Member

- A data member of a class can be qualified as static.
- Static variable are used to maintain values common to the entire class.

Characteristics

- It is initialized to zero when the first object of the class is created. No other initialization is permitted.
- Only one copy of that member is created for the entire class and is shared by all the objects of the class.
- It is visible only within the class but its lifetime is the entire program.

Example:

class item

```
{
    static int count;
    int number;
Public:
    void getdata(int a)
    {
        number = a;
        count++;
    }
    void getcount(void)
    {
        cout << "count:" << count;
    }
};
```

```
int item::count; // definition of the static data member
```

```
int main()
```

```
{
```

```
    item a,b,c; // count is initialized to zero
```

```
    a.getcount();
```

```
    b.getcount();
```

```
    c.getcount();
```

```
    a.getdata(100);
```

```
    b.getdata(200);
```

```
    c.getdata(300);
```

```
    cout << "After reading data" << "\n";
```

```
    a.getcount();
```

```
    b.getcount();
```

```
    c.getcount();
```

```
    return 0;
```

```
}
```

Output:

count: 0

count:0

count:0

After reading

count:3

count:3

count:3

Static Member Functions

- A static member function has following properties
- A static function can have access to only other static members (function or variable) declared in the same class
- A static member function can be called using the class name (instead of its objects) as follows:
 - `class name::fn-name;`

Class test

```
{
    int code;
    static int count;
    public:
        void setcode(void)
        {
            code = ++count;
        }
        void showcode(void)
        {
            cout << "obj member:" << code << "\n";
        }
}
```

```
static void showcount(void) // static member function
{
    cout << "count:" << count << "\n";
}

};

int test::count;

int main()
{
    test t1,t2;
    t1.setcode();
    t2.setcode();
    test::showcount();
    test t3;
    t3.setcode();
}
```

```
test::showcount();  
t1.showcode();  
t2.showcode();  
t3.showcode();  
return 0;  
}
```

Output:

count:2

count:3

obj number:1

obj number:2

obj number:3

Arrays Fundamentals

Array:

- An array is a collection of similar data items that are stored under a common name.
- A value in an array is identified by index or subscript enclosed in square brackets with array name.

Array Declaration:

Datatype arrayname[size];

eg: int a[10];

Types of Array:

- One-Dimensional Arrays
- Two- Dimensional Arrays
- Multi-Dimensional Arrays

Example:1 Read and display n numbers using an array

```
void main()
{
int a[10],l,n;
cout<<"Enter the number elements"<<endl;
cin>>n;
cout<<"Enter the  elements"<<endl;
for(i=0;i<5;i++)
cin>>a[i];
For(i=0;i<5;i++)
cout<<a[i];
}
```

Output:

Enter the number elements

5

Enter the elements

1

2

3

4

5

123456

Example:2 sorting

```
#include<iostream.h>
#include<conio.h>
void main()
{
int a[10],i,j,t;
clrscr();
for(i=0;i<=5;i++)
cin>>a[i];
for(i=0;i<=5;i++)
{
for(j=i+1;j<=5;j++)
{
```

```
if (a[i]>a[j])
{
t=a[i];
a[i]=a[j];
a[j]=t;
}
}
}
for(i=0;i<=5;i++)
cout<<a[i]<<endl;
getch();
}
```

Output:

5

2

3

1

6

4

1

2

3

4

5

6

String Operations

String:

- A string is sequence of characters enclosed within double quotes.
- Every string is automatically terminated by null character ie '\0'.
- String literals are enclosed within double quotes("A") whereas character literals are enclosed within single quotes('A').
- In C++language string type is not separately available and character pointers are used to represent strings.

Declaration of character array :

Datatype arrayname[size]=initializtional_list;

Eg:

1.char str[6] = "Hello";

2.char str[6]={ 'H','e','l','l','o','\0' }

Various String on Operations:

1. `strcpy(s1, s2);` → Copies string s2 into string s1.
2. `strcat(s1, s2);` → Concatenates string s2 onto the end of string s1.
3. `strlen(s1);` → Returns the length of string s1.
4. `strcmp(s1, s2);` → Returns 0 if s1 and s2 are the same; negative value if $s1 < s2$; Positive value if $s1 > s2$.
5. `toupper(ch);` -> Returns the upper case character of ch.
6. `tolower(ch);` -> Returns the lower case character of ch.
7. `strchr(s1, ch);` → Returns a pointer to the first occurrence of character ch in string s1.
8. `strstr(s1, s2);` → Returns a pointer to the first occurrence of string s2 in string s1.

Example:

```
#include <iostream>
#include <cstring>
using namespace std;
int main () {
    char str1[10] = "Hello";
    char str2[10] = "World";
    char str3[10];
    int len ;
    // copy str1 into str3
    strcpy( str3, str1);
    cout << "strcpy( str3, str1) : " << str3 << endl;
    // concatenates str1 and str2
    strcat( str1, str2);
    cout << "strcat( str1, str2): " << str1 << endl;
```

```
// total length of str1 after concatenation
    len = strlen(str1);
    cout << "strlen(str1) : " << len << endl;
    return 0;
}
```

Output:

```
strcpy( str3, str1) : Hello
strcat( str1, str2): HelloWorld
strlen(str1) : 10
```